

Sec 8.2. JavaScript Essentials

JavaScript Essentials

Task 1 - Introduction

JavaScript (JS): JavaScript is a scripting language used to add interactive and dynamic features to websites. It works together with **HTML** (structure) and **CSS** (styling) to create features such as form validation, click actions, animations, and other user interactions. JavaScript is mainly executed within web pages alongside HTML.

Task 2 - Essential Concepts

- **Variables:** Variables are containers used to store and reference data values in JavaScript. Each variable has a name that allows developers to access and use the stored value later. JavaScript provides three ways to declare variables: **var**, **let**, and **const**. `var` is function-scoped, while `let` and `const` are block-scoped, providing better control over variable access.
- **Data Types:** Data types define the kind of value a variable stores. Common JavaScript data types include **string** (text), **number**, **boolean** (true/false), **null**, **undefined**, and **object** (complex data such as arrays and objects).
- **Functions:** Functions are reusable blocks of code designed to perform specific tasks. They help organize code by grouping related actions together and can accept inputs called **arguments** to process data.
- **Loops:** Loops allow developers to execute the same block of code multiple times while a condition is true. Common JavaScript loops include **for**, **while**, and **do...while**, which are useful for repeating tasks such as processing lists or performing actions on multiple items.

Answer the questions below

What term allows you to run a code block multiple times as long as it is a condition? loop

Task 3 - JavaScript Overview

JS is an **interpreted** language, meaning the code is executed directly in the browser without prior compilation. Below is a sample JS code demonstrating key concepts, such as **defining a variable**, **understanding data types**, **using control flow statements**, and writing simple functions.

```
// Hello, World! program
console.log("Hello, World!");

// Variable and Data Type
let age = 25; // Number type

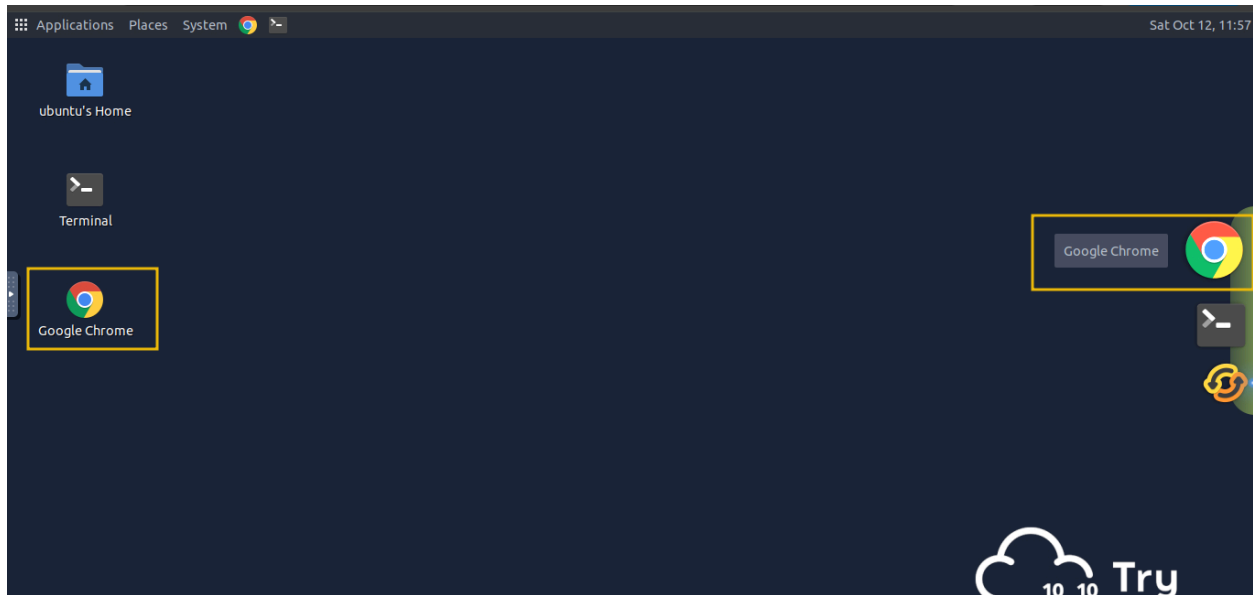
// Control Flow Statement
if (age >= 18) {
    console.log("You are an adult.");
} else {
    console.log("You are a minor.");
}

// Function
function greet(name) {
    console.log("Hello, " + name + "!");
}

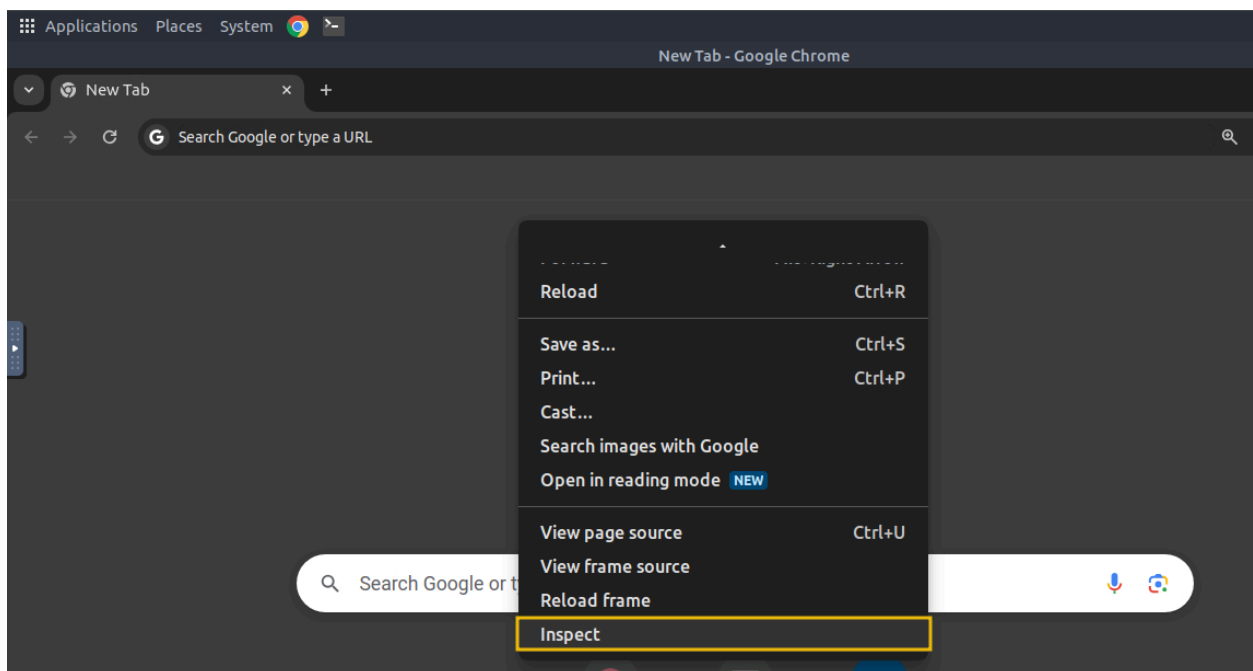
// Calling the function
greet("Bob");
```

JS is primarily executed on the client side, which makes it easy to inspect and interact with HTML directly within the browser. We'll use the `Google Chrome Console` feature to run our first JS program, allowing us to write and execute JS code easily without additional tools. Follow these steps to get started:

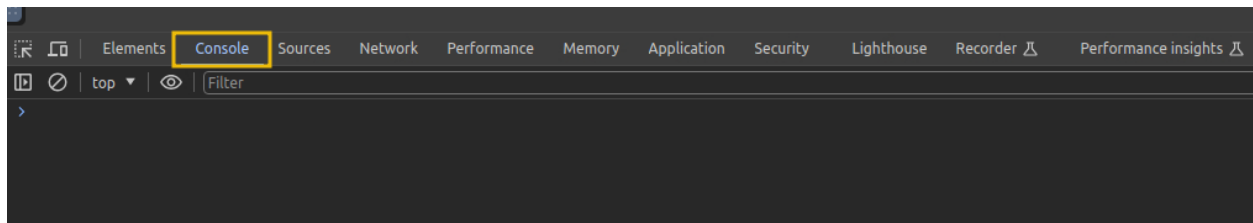
- Open `Google Chrome` by clicking the `Google Chrome` icon on the Desktop of the VM.



- Once Chrome is open, press `Ctrl + Shift + I` to open the `Console` or right-click anywhere on the page and select `Inspect`.



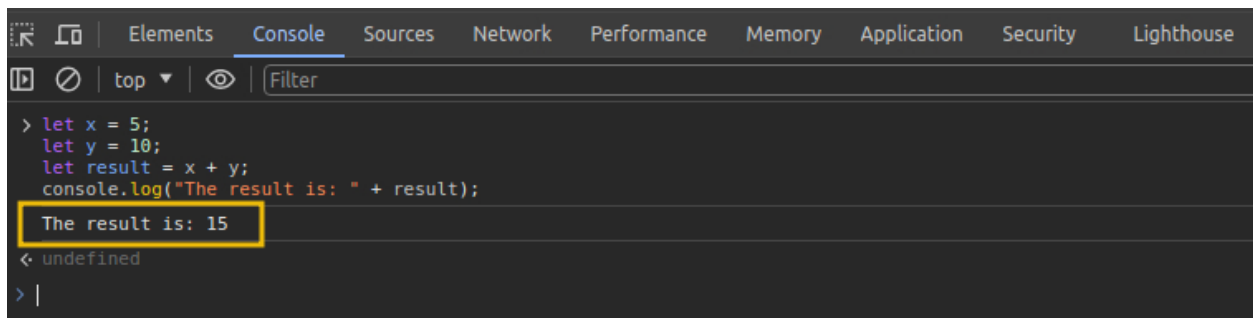
- Then, click on the `Console` tab. This console allows you to run JS code directly in the browser without installing additional software.



- Let's create a simple JS program that adds two numbers and displays the result. Below is the code:

```
let x = 5;  
let y = 10;  
let result = x + y;  
console.log("The result is: " + result);
```

- In the code above, `x` and `y` are variables holding the numbers. `x + y` is an expression that adds the two numbers together, whereas `console.log` is a function used to print the result to the console.
- Copy the above code and paste it into the console by pressing the key `Ctrl + V`. Once pasted, press `Enter`. You should see the result displayed as:



Answer the questions below

What is the code output if the value of `x` is changed to 10? The result is: 20

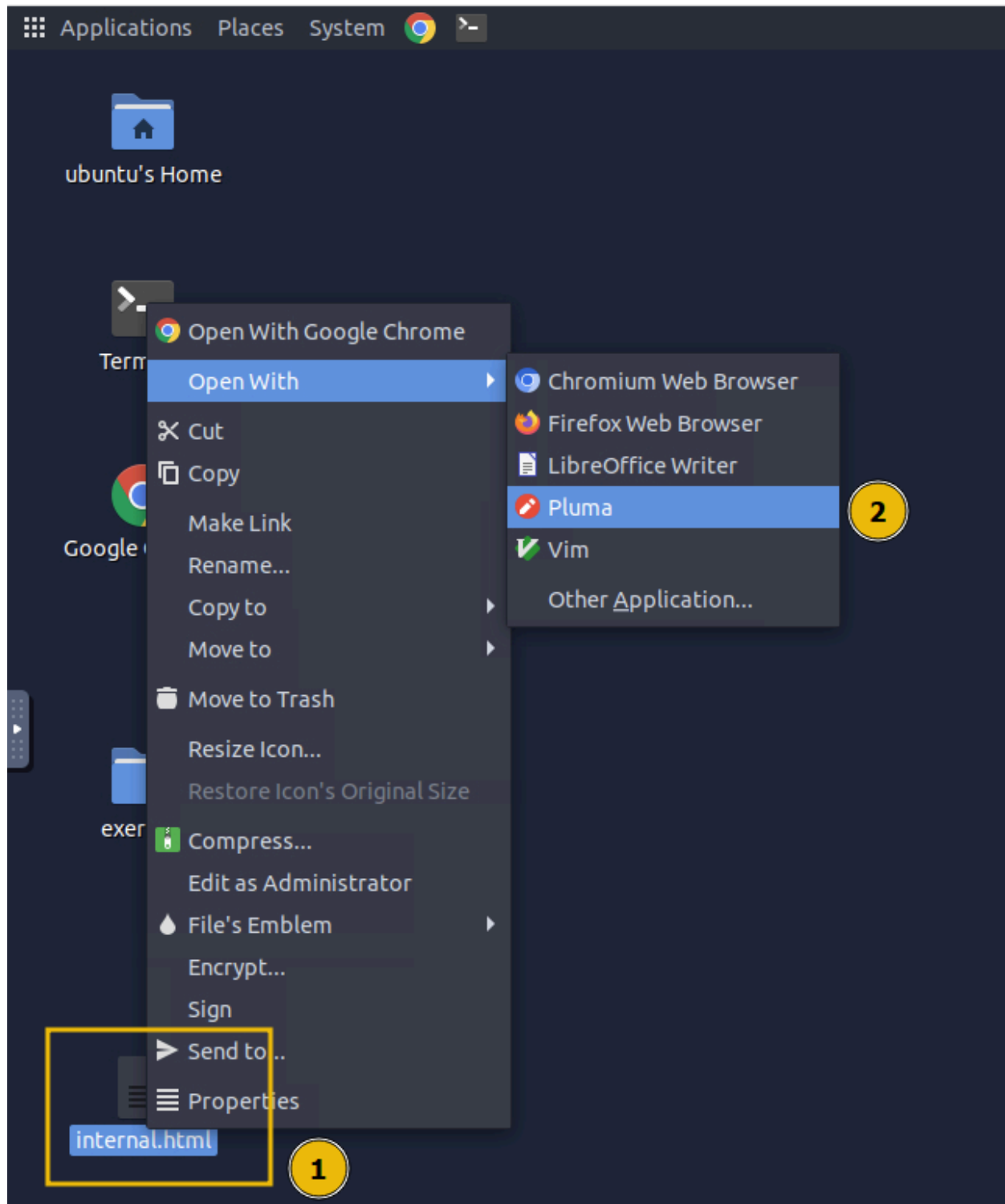
Is JavaScript a compiled or interpreted language? Interpreted

Task 4 - Integrating JavaScript in HTML

JS is not used to render content; it works with HTML and CSS to create dynamic and interactive web pages. There are two main ways to integrate JS into HTML: internally and externally.

Internal JavaScript: Internal JavaScript is when JavaScript code is written directly inside an HTML document using `<script>` **tags**. It is commonly used by beginners because it makes it easier to understand how JavaScript interacts with HTML elements. Scripts can be placed in the `<head>` **section** to load before the webpage content or in the `<body>` **section** to interact with elements after they are loaded.

Example: To create an HTML document with internal JS, right-click on the `Desktop` and select `Create Document > Empty File`. Name the file `internal.html`. Next, right-click the `internal.html` file and `choose Open with Pluma` to open it in a text editor.



Once the editor is open, paste the following code:

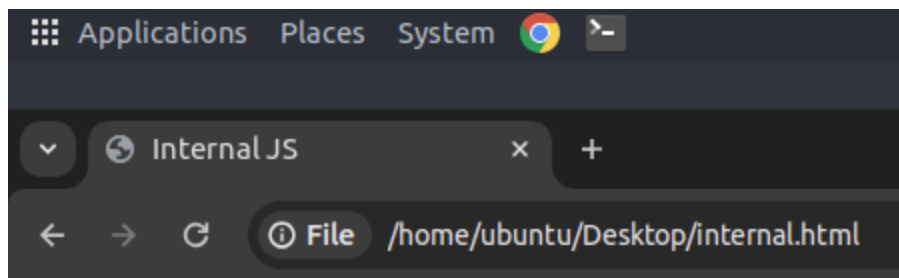
```

<!DOCTYPE html>
<html lang="en">
<head>
  <title>Internal JS</title>
</head>
<body>
  <h1>Addition of Two Numbers</h1>
  <p id="result"></p>

  <script>
    let x = 5;
    let y = 10;
    let result = x + y;
    document.getElementById("result").innerHTML = "The re
sult is: " + result;
  </script>
</body>
</html>

```

After pasting the code, click **File** and select **Save**, which will save the file to `internal.html`. Double-click the file to open it in Chrome browser, where you will see the following output:



Addition of Two Numbers

The result is: 15

In this HTML document, we are using internal JS, meaning the code is placed directly inside the HTML file within the **<script>** tag. The script performs a simple task: it adds two numbers (x and y) and then displays the result on the web page. The JS interacts with the HTML by selecting an element (**<p> with id="result"**) and updating its content using `document.getElementById("result").innerHTML`. This internal JS is executed when the browser loads the HTML file.

External JavaScript: External JS involves creating and storing JS code in a separate file ending with a `.js` file extension. This method helps developers keep the HTML document clean and organised. The external JS file can be stored or hosted on the same web server as the HTML document or stored on an external web server such as the cloud. First, create a new file named `script.js` and save it on the `Desktop` with the following code:

```
let x = 5;
let y = 10;
let result = x + y;
document.getElementById("result").innerHTML = "The result is:
" + result;
```

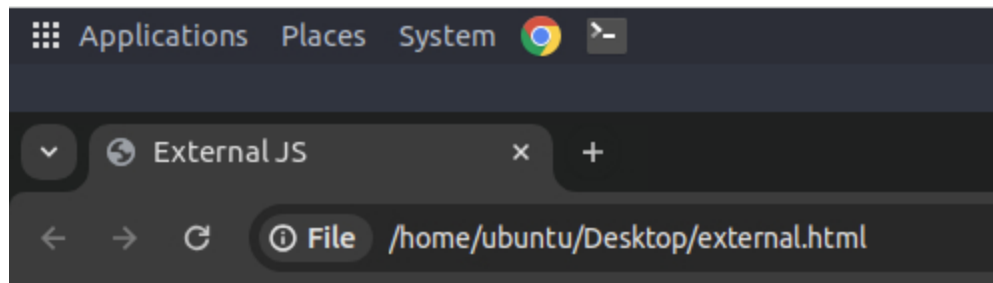
Next, create a new file named `external.html` and paste the following code (notice that the HTML code is the same as that of the previous example):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>External JS</title>
</head>
<body>
  <h1>Addition of Two Numbers</h1>
  <p id="result"></p>

  <!-- Link to the external JS file -->
```

```
<script src="script.js"></script>
</body>
</html>
```

Now, double-click the external.html file and check the results. Do you see any difference? No, the output remains the same as in the previous example.

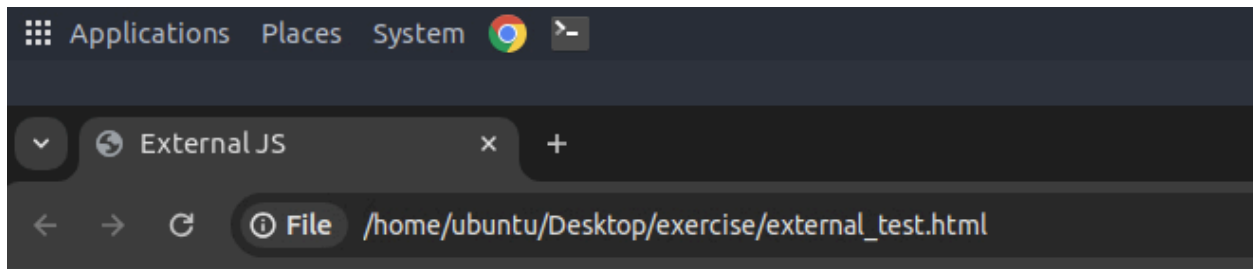


Addition of Two Numbers

The result is: 15

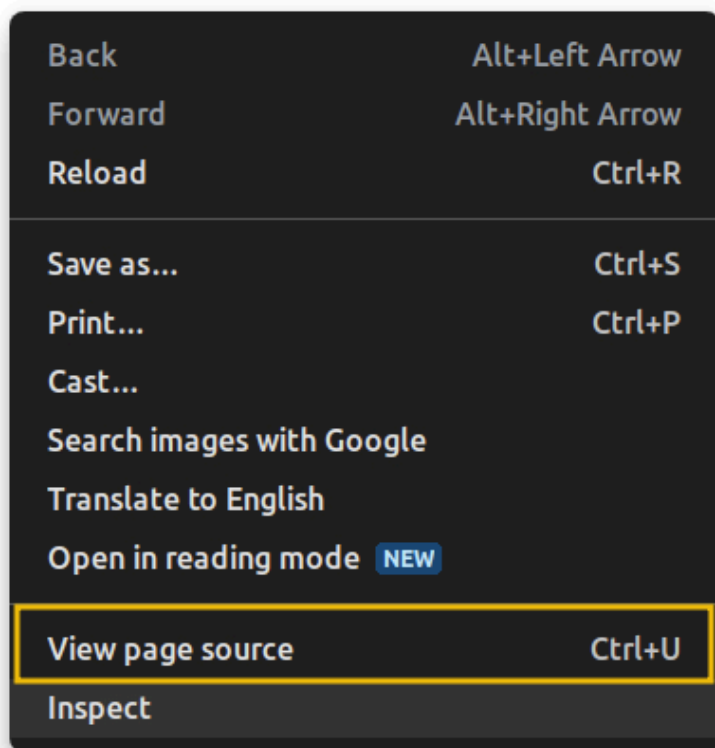
What we did differently is use the `src` attribute in the `<script>` tag to load the JS from an external file. When the browser loads the page, it looks for the `script.js` file and loads its content into the HTML document. This approach allows us to keep the JS code separate from the HTML, making the code more organised and easier to maintain, especially when working on larger projects.

Verifying Internal or External JS: When pen-testing a web application, it is important to check whether the website uses internal or external JS. This can be easily verified by viewing the page's source code. To do this, open the page `external_test.html` located in the `exercise` folder in `Chrome`, right-click anywhere on the page, and select `View Page Source`.

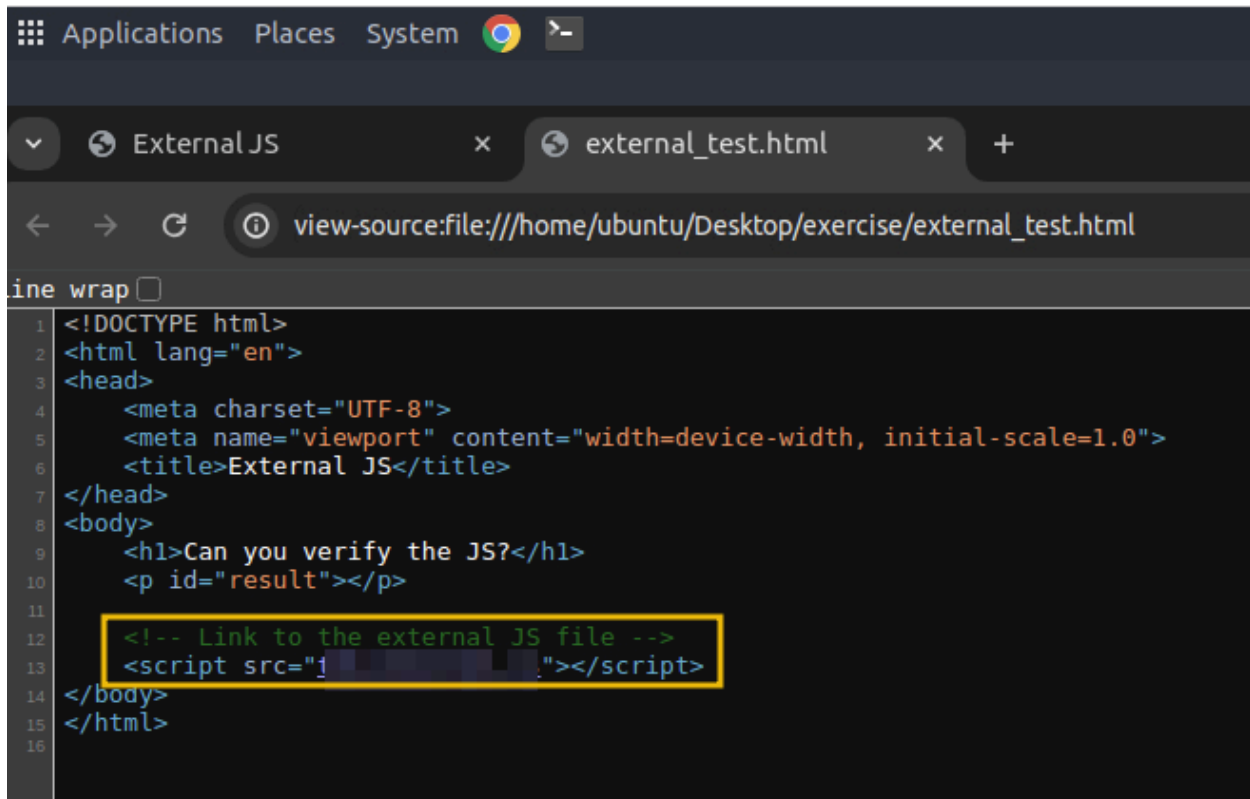


Can you verify the JS?

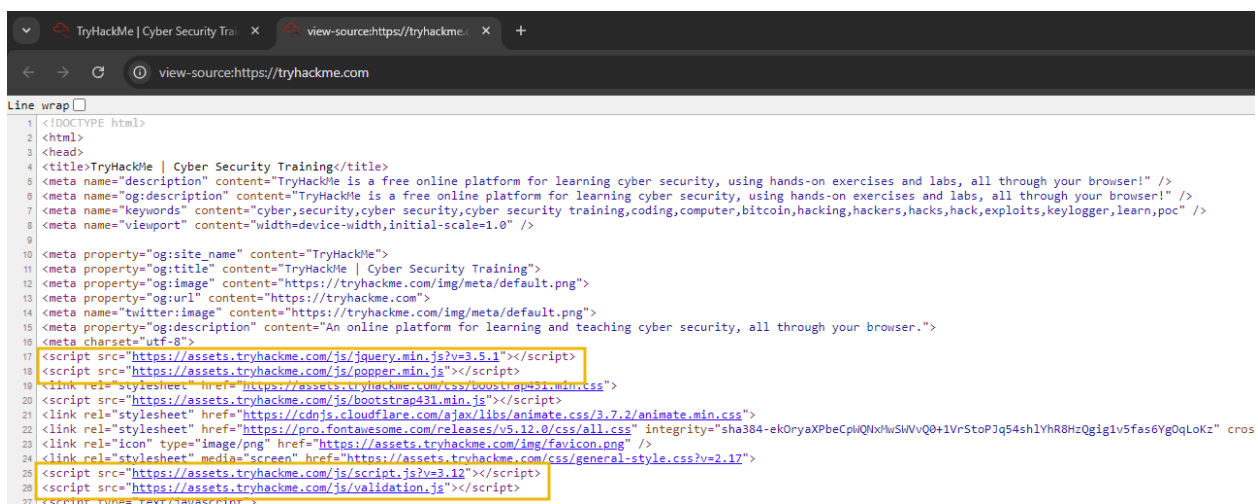
Hint: Wanna check the source?



This will display the HTML code of the rendered page. Inside the source code, any JS written directly on the page will appear between **<script>** tags without the `src` attribute. If you see a **<script>** tag with a `src` attribute, it indicates that the page is loading external JS from a separate file.



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>External JS</title>
7 </head>
8 <body>
9   <h1>Can you verify the JS?</h1>
10  <p id="result"></p>
11
12  <!-- Link to the external JS file -->
13  <script src="1" ></script>
14 </body>
15 </html>
16
```



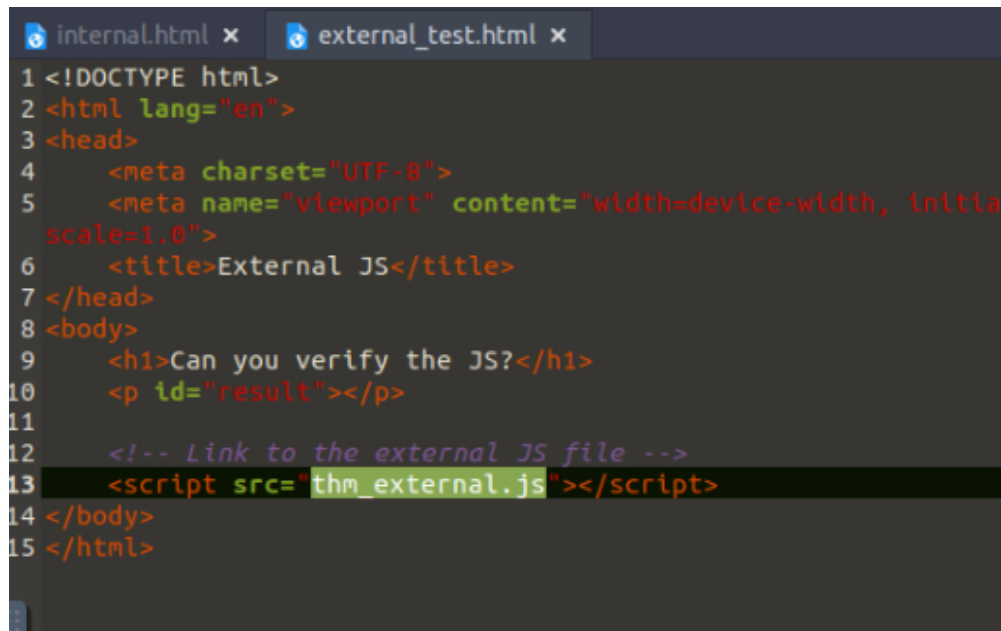
```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <title>TryHackMe | Cyber Security Training</title>
5 <meta name="description" content="TryHackMe is a free online platform for learning cyber security, using hands-on exercises and labs, all through your browser!" />
6 <meta name="og:description" content="TryHackMe is a free online platform for learning cyber security, using hands-on exercises and labs, all through your browser!" />
7 <meta name="keywords" content="cyber,security,cyber security,cyber security training,coding,computer,bitcoin,hacking,hackers,hacks,hack,exploits,keylogger,learn,poc" />
8 <meta name="viewport" content="width=device-width,initial-scale=1.0" />
9
10 <meta property="og:site_name" content="TryHackMe">
11 <meta property="og:title" content="TryHackMe | Cyber Security Training">
12 <meta property="og:image" content="https://tryhackme.com/img/meta/default.png">
13 <meta property="og:url" content="https://tryhackme.com">
14 <meta name="twitter:image" content="https://tryhackme.com/img/meta/default.png">
15 <meta property="og:description" content="An online platform for learning and teaching cyber security, all through your browser.">
16 <meta charset="utf-8">
17 <script src="https://assets.tryhackme.com/js/jquery.min.js?v=3.5.1"></script>
18 <script src="https://assets.tryhackme.com/js/popper.min.js"></script>
19 <link rel="stylesheet" href="https://assets.tryhackme.com/css/bootstrap.min.css">
20 <script src="https://assets.tryhackme.com/js/bootstrap431.min.js"></script>
21 <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/animate.css/3.7.2/animate.min.css">
22 <link rel="stylesheet" href="https://pro.fontawesome.com/releases/v5.12.0/css/all.css" integrity="sha384-ekOryaXPbeCpWQ0+1VrStoPjg54sh1YhR8HzQig1v5fas6YgQoLoKz" cross
23 <link rel="icon" type="image/png" href="https://assets.tryhackme.com/img/favicon.png" />
24 <link rel="stylesheet" media="screen" href="https://assets.tryhackme.com/css/general-style.css?v=2.17">
25 <script src="https://assets.tryhackme.com/js/script.js?v=3.12"></script>
26 <script src="https://assets.tryhackme.com/js/validation.js"></script>
27 <script src="https://assets.tryhackme.com/js/validation.js"></script>
```

Answer the questions below

Which type of JavaScript integration places the code directly within the HTML document? Internal

Which method is better for reusing JS across multiple web pages? External

What is the name of the external JS file that is being called by external_test.html? thm_external.js



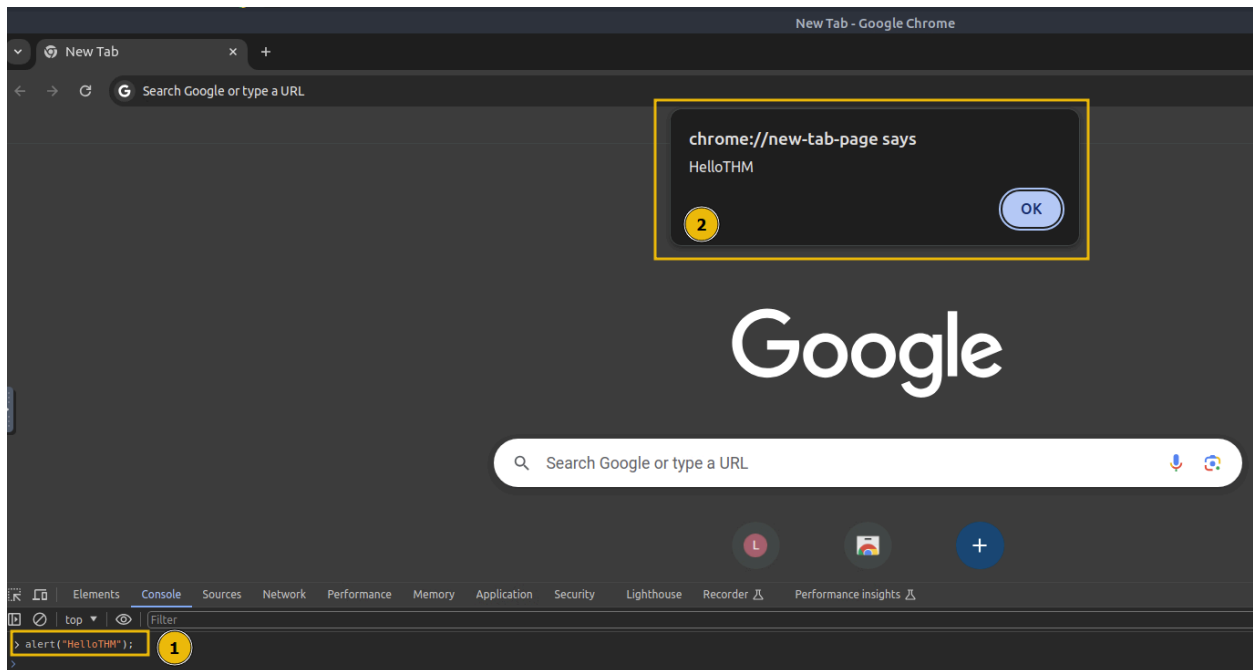
```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>External JS</title>
7 </head>
8 <body>
9   <h1>Can you verify the JS?</h1>
10  <p id="result"></p>
11
12  <!-- Link to the external JS file -->
13  <script src="thm_external.js"></script>
14 </body>
15 </html>
```

What attribute links an external JS file in the <script> tag? src

Task 5 - Abusing Dialogue Functions

JS provides built-in functions like `alert`, `prompt`, and `confirm` to facilitate this interaction. These functions allow developers to display messages, gather input, and obtain user confirmation.

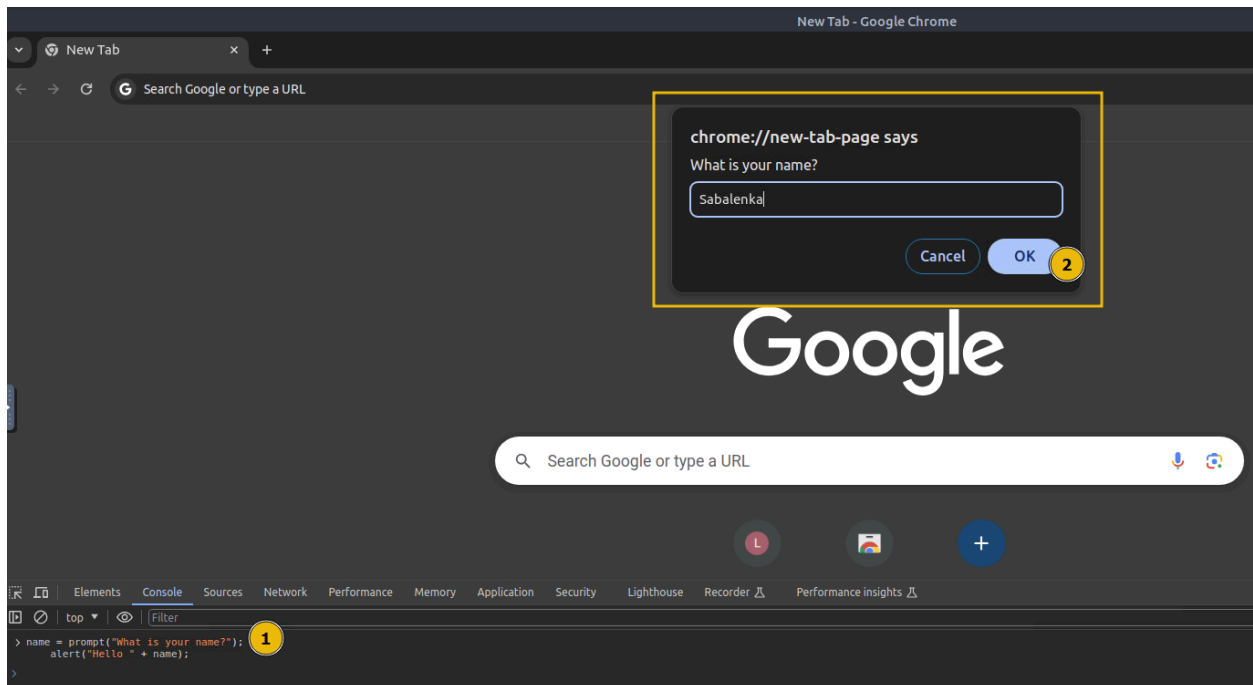
Alert: The alert function displays a message in a dialogue box with an "OK" button, typically used to convey information or warnings to users. For example, if we want to display "Hello THM" to the user, we would use an `alert("HelloTHM");`. To try it out, open the Chrome console, type `alert("Hello THM")`, and press Enter. A dialogue box with the message will appear on the screen.



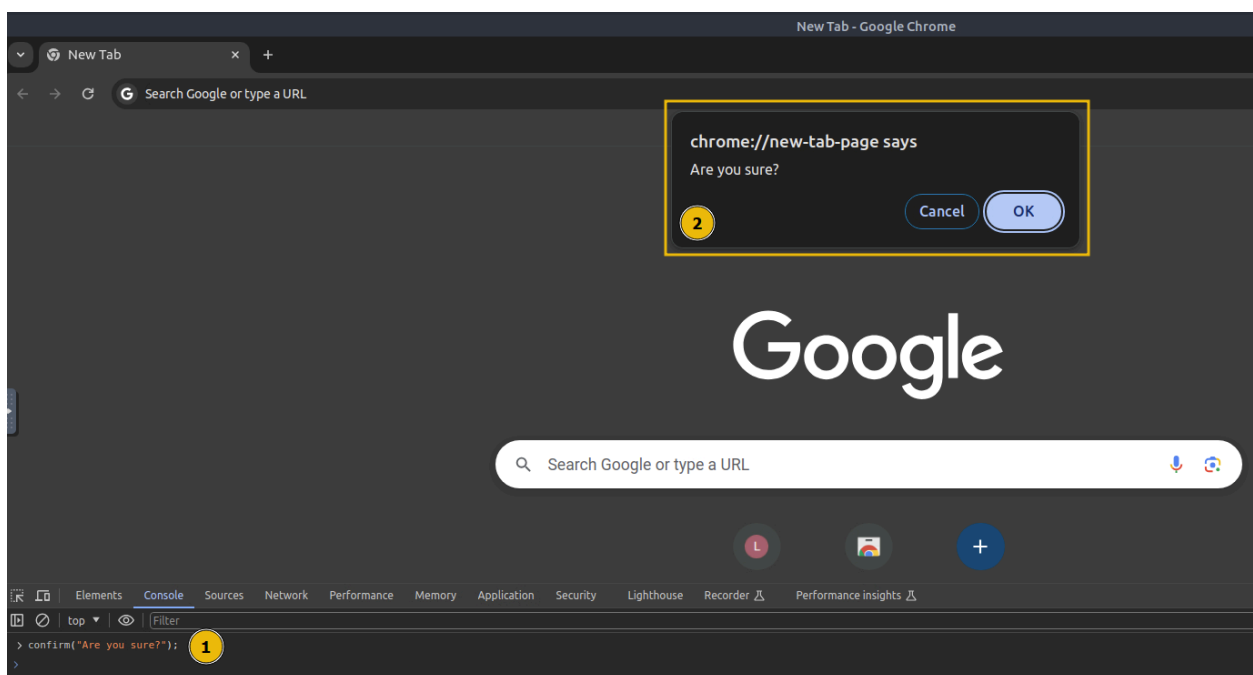
Prompt: The prompt function displays a dialogue box that asks the user for input. It returns the entered value when the user clicks "OK", or null if the user clicks "Cancel". For example, to ask the user for their name, we would use `prompt("What is your name?");`. To test this, open the Chrome console and paste the following that asks for a username and then greets him.

```
name = prompt("What is your name?");  
alert("Hello " + name);
```

Once you paste the code and hit Enter, a dialogue box will appear, and the value entered by the user will be returned to the console.



Confirm: The confirm function displays a dialogue box with a message and two buttons: "OK" and "Cancel". It returns true if the user clicks "OK" and false if the user clicks "Cancel". For example, to ask the user for confirmation, we would use `confirm("Are you sure?");`. To try this out, open the Chrome console, type `confirm("Do you want to proceed?")`, and press `Enter`.



A dialogue box will appear, and depending on whether the user clicks "OK" or "Cancel", the value true or false will be returned to the console.

Answer the questions below

In the file invoice.html, how many times does the code show the alert Hacked?
5

Which of the JS interactive elements should be used to display a dialogue box that asks the user for input? prompt

If the user enters Tesla, what value is stored in the carName= prompt("What is your car name?")? in the carName variable? Tesla

Task 6 - Bypassing Control Flow Statements

Control flow in JS refers to the order in which statements and code blocks are executed based on certain conditions. JS provides several control flow structures such as `if-else`, `switch` statements to make decisions, and loops like `for`, `while`, and `do...while` to repeat actions. Proper use of control flow ensures that a program can handle various conditions effectively.

Conditional Statements in Action: One of the most used conditional statements is the `if-else` statements, which allows you to execute different blocks of code depending on whether a condition evaluates to `true` or `false`. To test this practically, let's create a file `age.html` on the Desktop of the attached VM and paste the following code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Age Verification</title>
</head>
<body>
  <h1>Age Verification</h1>
  <p id="message"></p>

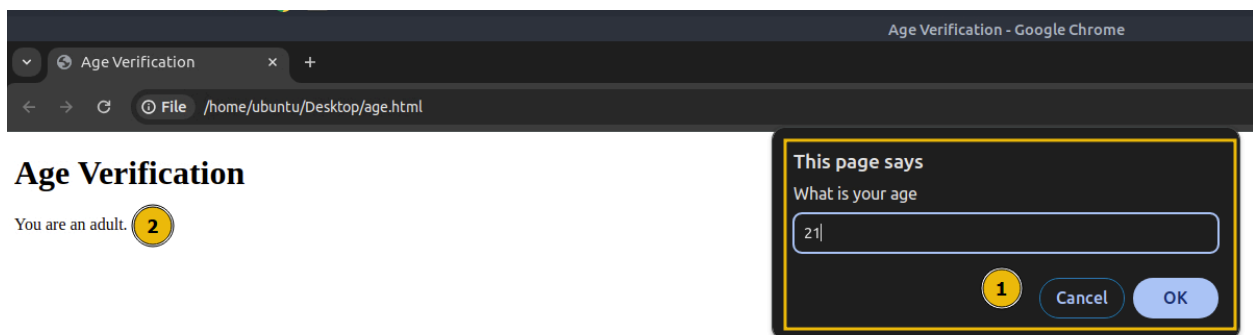
  <script>
```

```

    age = prompt("What is your age")
    if (age >= 18) {
        document.getElementById("message").innerHTML = "You are an adult.";
    } else {
        document.getElementById("message").innerHTML = "You are a minor.";
    }
</script>
</body>
</html>

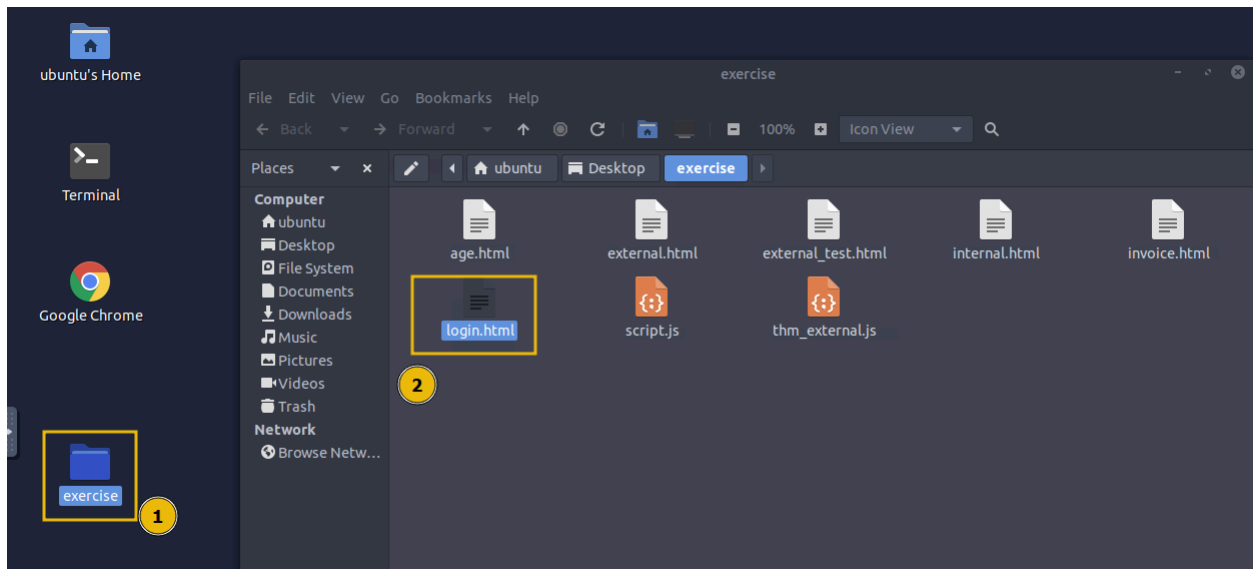
```

Double-click the file to open it in Google Chrome. You will see the following image:

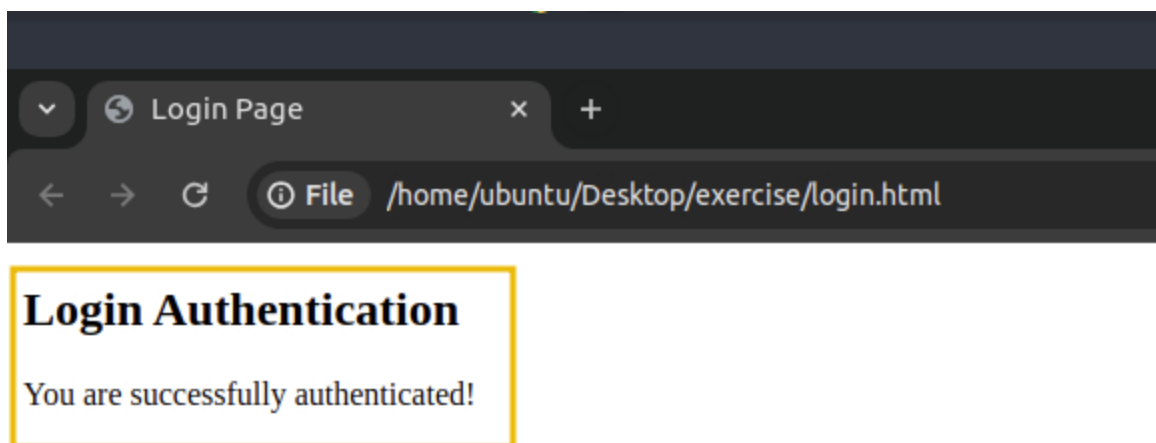


In the above code, a prompt will ask for your age. If your age is greater than or equal to 18, it will display a message saying, "You are an adult." Otherwise, it will show a different message. This behaviour is controlled by the if-else statement, which checks the value of the age variable and displays the appropriate message based on the condition.

Bypassing Login Forms: Suppose a developer has implemented authentication functionality in JS, where only users with the username "admin" and passwords matching a specific value are allowed to log in. To see this in action, open the `login.html` file in the exercises folder.



When you double-click the file and open it in your browser, it will prompt you for a username and password. If the correct credentials are entered, it will display a message confirming that you are logged in, as shown below:



Answer the questions below

What is the message displayed if you enter the age less than 18? You are a minor.

What is the password for the user admin? ComplexPassword

Task 7 - Exploring Minified Files

- **JavaScript Minification:** Minification is the process of reducing the size of JavaScript files by removing unnecessary characters such as spaces, line breaks, comments, and shortening variable names. It improves website performance by making files smaller and faster to load while keeping the same functionality.
- **JavaScript Obfuscation:** Obfuscation makes JavaScript code harder to understand by renaming variables and functions, adding unnecessary code, and inserting confusing elements. Unlike minification, its main purpose is to hide the code logic and make reverse engineering more difficult.

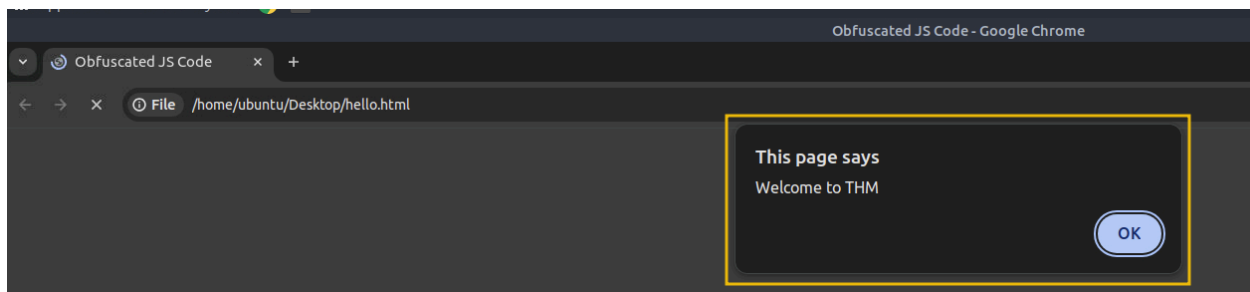
Practical Example: Create a file on the Desktop of the attached VM with the name `hello.html` and paste the following HTML code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Obfuscated JS Code</title>
</head>
<body>
  <h1>Obfuscated JS Code</h1>
  <script src="hello.js"></script>
</body>
</html>
```

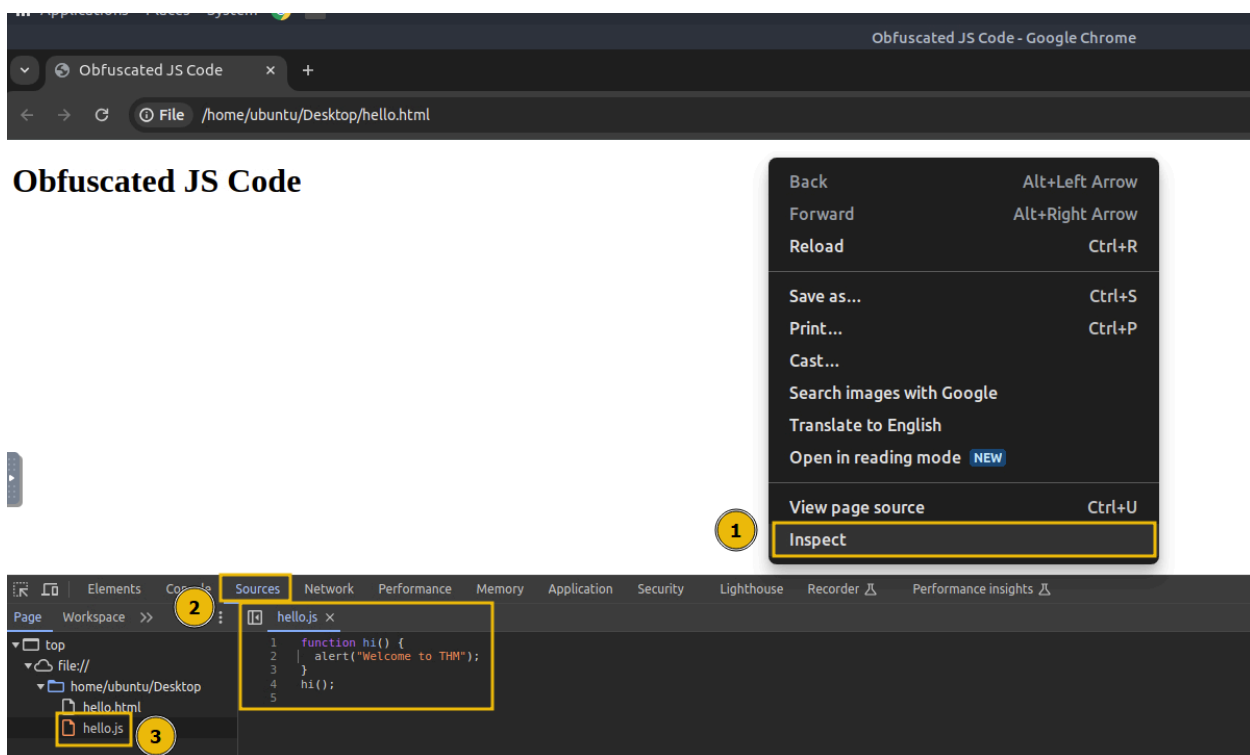
Then, create another file with the name `hello.js` and add the following code:

```
function hi() {
  alert("Welcome to THM");
}
hi();
```

Now, double-click the `hello.html` file to open it in Google Chrome. Once the file is opened, you will see an alert greeting you with "`Welcome to THM`".

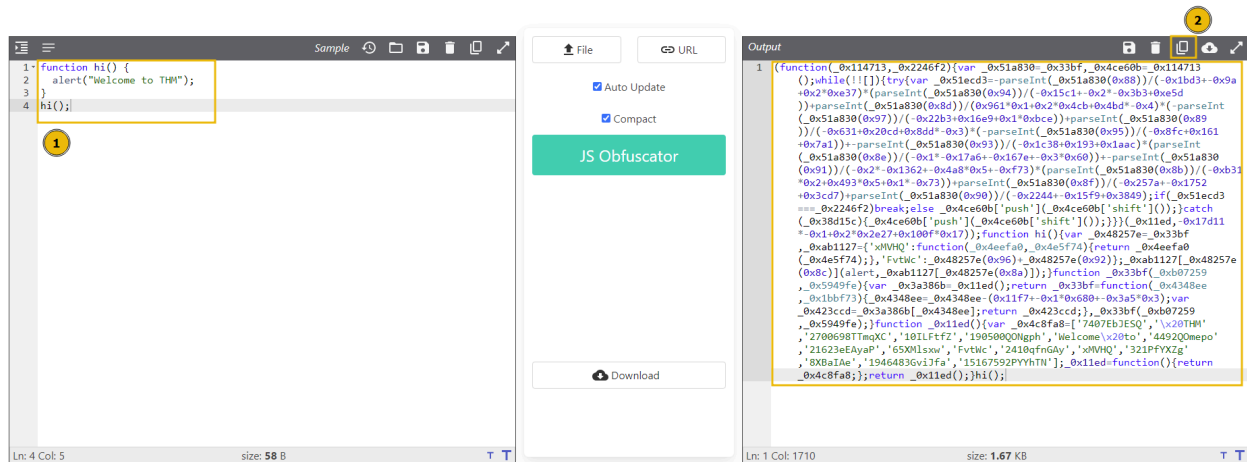


Click **OK** to close the alert dialogue box. Right-click anywhere on the page and click on **Inspect** to open the developer tools. In the developer tools, navigate to the **Sources** tab and click on the **hello.js** file to view the source code. You will see that the JS code is easily accessible and viewable, as shown below:



Obfuscation in Action

Now, we will try to minify and obfuscate the JS code using an online tool. Visit the [website \(opens in new tab\)](#) and copy the contents of **hello.js**, and paste them into the dialogue box on the website. The tool will minify and obfuscate the code, turning it into a string of gibberish characters shown below:



But what if we tell you that these gibberish characters are still fully functional code? The only difference is that they are not human-readable, but the browser can still execute them correctly. The website converted our JS code to this one:

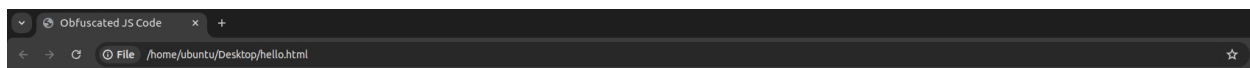
```
(function(_0x114713,_0x2246f2){var _0x51a830=_0x33bf,_0x4ce60b=_0x114713();while(![]){try{var _0x51ecd3=-parseInt(_0x51a830(0x88))/(-0x1bd3+-0x9a+0x2*0xe37)*(parseInt(_0x51a830(0x94))/(-0x15c1+-0x2*-0x3b3+0xe5d))+parseInt(_0x51a830(0x8d))/(0x961*0x1+0x2*0x4cb+0x4bd*-0x4)*(-parseInt(_0x51a830(0x97))/(-0x22b3+0x16e9+0x1*0xbce))+parseInt(_0x51a830(0x89))/(-0x631+0x20cd+0x8dd*-0x3)*(-parseInt(_0x51a830(0x95))/(-0x8fc+0x161+0x7a1))+parseInt(_0x51a830(0x93))/(-0x1c38+0x193+0x1aac)*(parseInt(_0x51a830(0x8e))/(-0x1*-0x17a6+-0x167e+-0x3*0x60))+parseInt(_0x51a830(0x91))/(-0x2*-0x1362+-0x4a8*0x5+-0xf73)*(parseInt(_0x51a830(0x8b))/(-0xb31*0x2+0x493*0x5+0x1*-0x73))+parseInt(_0x51a830(0x8f))/(-0x257a+-0x1752+0x3cd7)+parseInt(_0x51a830(0x90))/(-0x2244+-0x15f9+0x3849);if(_0x51ecd3===_0x2246f2)break;else _0x4ce60b['push'](_0x4ce60b['shift'])}}catch(_0x38d15c){_0x4ce60b['push'](_0x4ce60b['shift'])}});function hi(){var _0x48257e=_0x33bf,_0xab1127=('xMHQ':function(_0x4eefa0,_0x4e5f74){return _0x4eefa0(_0x4e5f74)}),_0x48257e(0x96)=_0x48257e(0x92);_0xab1127[_0x48257e(0x8c)](alert,_0xab1127[_0x48257e(0x8a)]);function _0x33bf(_0x607259,_0x5949fe){var _0x3a386b=_0x11ed;return _0x33bf=function(_0x4348ee,_0x1bbf73){_0x4348ee=_0x4348ee(_0x11f+-0x1*0x680+-0x3a5*0x3);var _0x423ccd=_0x3a386b[_0x4348ee];return _0x423ccd;},_0x33bf(_0x607259,_0x5949fe);function _0x11ed(){var _0x4c8fab=('7407Eb3E5Q','x20THM','2700698TmxQC','10L1PtF2','190500Q0igph','Welcome to THM','449Q0mepo','21623eAayap','65X0lsmu','FvtWc','2418qfnGay','xMHQ','321PFYXZg','8XBaIAe','1946483Gvi7fa','15167592PYyHTN');_0x11ed=function(){return _0x4c8fab;};return _0x11ed();};hi();}
```

```

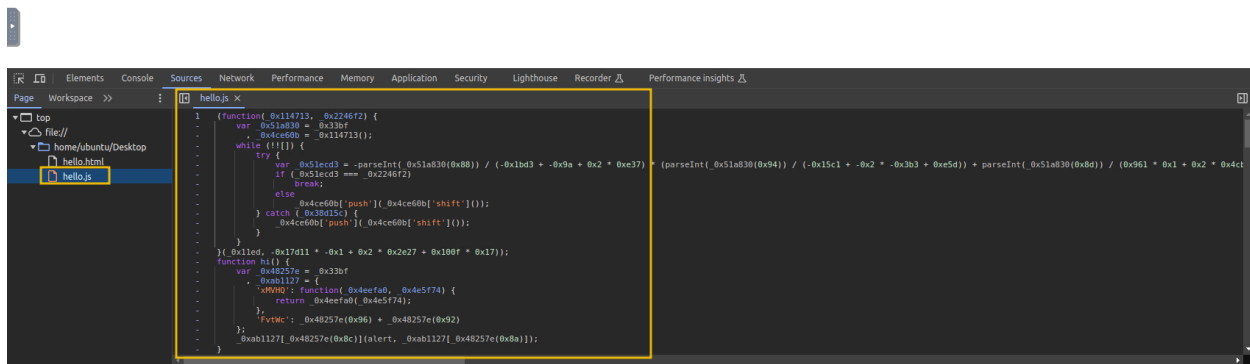
( ));}catch(_0x38d15c)
{ _0x4ce60b['push'](_0x4ce60b['shift']());}})(_0x11ed,-0x17d11
*-0x1+0x2*0x2e27+0x100f*0x17));
function hi(){var _0x48257e=_0x33bf,_0xab1127={'xMVHQ':function(_0x4eefa0,_0x4e5f74)
{return _0x4eefa0(_0x4e5f74);}},'FvtWc':_0x48257e(0x96)+_0x482
57e(0x92)};_0xab1127[_0x48257e(0x8c)
](alert,_0xab1127[_0x48257e(0x8a)]);}function _0x33bf(_0xb072
59,_0x5949fe){var _0x3a386b
=_0x11ed();return _0x33bf=function(_0x4348ee,_0x1bbf73){_0x43
48ee=_0x4348ee-(0x11f7+-
0x1*0x680+-0x3a5*0x3);var _0x423ccd=_0x3a386b[_0x4348ee];retu
rn _0x423ccd;},_0x33bf
(_0xb07259,_0x5949fe);}function _0x11ed(){var _0x4c8fa8=['740
7EbJESQ','\x20THM',
'2700698TTmqXC','10ILFtfZ','190500Q0Ngph','Welcome\x20to',
'4492Q0mepo','21623eEAyaP','65XMlsxw','FvtWc','2410qfnGAy','x
MVHQ','321PfYXZg',
'8XBaIAe','1946483GviJfa','15167592PYYhTN'];_0x11ed=function
(){return _0x4c8fa8;};return _0x11ed();}hi();

```

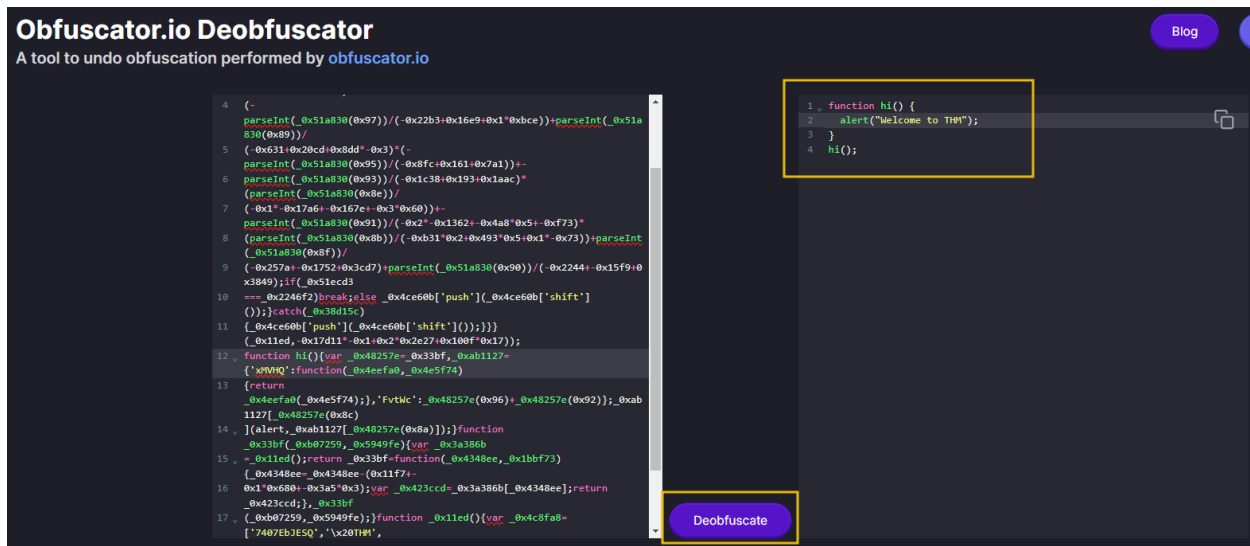
Click the **Copy to Clipboard** (highlighted as **2** in the above image) button as shown on the website. Then, remove the current content of `hello.js` on the attached VM and paste the obfuscated content into the file. Reload the `hello.html` file in Google Chrome and inspect the source code again under the **Sources** tab. You will notice that the code is now obfuscated but still functions exactly the same as before.



Obfuscated JS Code



Deobfuscating a Code: We can also deobfuscate an obfuscated code using an online tool. Visit the [website \(opens in new tab\)](#), then paste the obfuscated code into the provided dialogue box. The website will generate the equivalent, human-readable JS code for you, making it easier to understand and analyze the original script.



Answer the questions below

What is the alert message shown after running the file hello.html? Welcome to THM

What is the value of the age variable in the following obfuscated code snippet?
`age=0x10x247e+0x35-0x2e+-0x1ae3; 21`

Task 8 - Best Practices

- **Client-side validation:** Never trust it alone; validate on the server.
- **Third-party libraries:** Use only trusted and updated libraries.
- **Hardcoded secrets:** Keep credentials on the server, not in JavaScript.
- **Minification & obfuscation:** Improve performance and make code harder to reverse-engineer.

Answer the questions below

Is it a good practice to blindly include JS in your code from any source (yea/nay)? nay